RDS - EX3

Yonatan Cohen - 324842608

January 2026

1 Payment Scheme

Settings:

- $n = 2f + 1$ servers
- f omission failures
- Synchronous model, delay bounded by Δ
- U is set of users, some may be malicious
- Each user $u \in U$ has a public key pk_u and a secret key sk_u
- $\sigma = sign(m, sk) = H(m)^{sk}$
- The system wide public key PK is known to all, secret key $SK = g(0)$ where g is uniformly random shared polynomial of degree at most f
- Each server i holds a share $s_i = g(i)$, and $g(0)$ is the secret shared among the servers
- A *token* is a triple $T = (P, v, \sigma)$ where P is a public key, v is a non-negative integer value, and σ is a signature
- A token $T = (P, v, \sigma)$ is valid if its signature is a valid signature on its public key and value, relative to the system wide public key PK - $verify((P, v), \sigma, PK) = 1$, i.e., the servers collectively authorize it by producing or validating a signature under the shared secret $g(0)$
- A token is considered unspent if it has not been previously consumed by a successful payment operation
- The system stores for each user the value of its un-minted balance
- Each user starts with a balance of 100

1.1 Protocol Pseudo-code

Algorithm 1 User

Initial State For User u:

$b = 100$ ▷ The initial balance
$T = \emptyset$ ▷ List of available tokens

Mint Request

Generate a new key pair (sk_u, pk_u)
Choose a value $v \leq b$
$b = b - v$
Send to all servers (pk_u, v, σ_u) where $\sigma_u = sign((pk_u, v), sk_u)$
Wait for $f + 1$ valid signatures shares
Using Lagrange interpolation in the exponent, reconstruct the shares to a final signature $\sigma = H(pk_u, v)^{g(0)}$
Add token (pk_u, v, σ) to T

Interactive Pay

Take two tokens $T_1, T_2 \in T$ s.t. $T_1 = (pk_{u_1}, v_1, \sigma_1)$ and $T_2 = (pk_{u_2}, v_2, \sigma_2)$ (or $T_2 = \perp$)
Get recipient public key and value (pk_a, v_a)
Get own public key and value for change (pk_u, v_u) (or $(\perp, \perp)$)
So the transaction is $tx = (T_1, T_2, (pk_a, v_a), (pk_u, v_u))$
Sign tx with sk_{u_1} and sk_{u_2} to get σ_{u_1} and σ_{u_2}
Send to all servers $(T_1, T_2, (pk_a, v_a), (pk_u, v_u), \sigma_{u_1}, \sigma_{u_2})$
Wait for 2 (1 if $(pk_u, v_u) = \perp$) valid signatures shares from at least $f + 1$ servers
Using Lagrange interpolation in the exponent, reconstruct the shares to a final signatures $\sigma_a = H(pk_a, v_a)^{g(0)}$ and $\sigma_u = H(pk_u, v_u)^{g(0)}$
Add token (pk_u, v_u, σ_u) to T and send (pk_a, v_a, σ_a) to a

Non-Interactive Pay

Take two tokens $T_1, T_2 \in T$ s.t. $T_1 = (pk_{u_1}, v_1, \sigma_1)$ and $T_2 = (pk_{u_2}, v_2, \sigma_2)$ (or $T_2 = \perp$)
Get recipient public key and value (pk_a, v_a)
Get own public key and value for change (pk_u, v_u) (or $(\perp, \perp)$)
So the transaction is $tx = (T_1, T_2, (pk_a, v_a), (pk_u, v_u))$
Sign tx with sk_{u_1} and sk_{u_2} to get σ_{u_1} and σ_{u_2}
Send to all servers $(T_1, T_2, (pk_a, v_a), (pk_u, v_u), \sigma_{u_1}, \sigma_{u_2})$
if $(pk_u, v_u) \neq (\perp, \perp)$ (need to receive change) **then**
    Wait for $f + 1$ valid signatures shares
    Using Lagrange interpolation in the exponent, reconstruct the shares to a final signature $\sigma_u = H(pk_u, v_u)^{g(0)}$
    Add token (pk_u, v_u, σ_u) to T
end if

Algorithm 2 Server

Initial State For Server i:

$B[u] = 100$ for all $u \in U$ ▷ Balance for each user
$M = \emptyset$ ▷ Nullifier set of used public keys to prevent replay attack
$N = \emptyset$ ▷ Nullifier set of spent tokens
$s_i = g(i)$ ▷ The system secret's share

Upon Receiving (pk_u, v, σ) **Where** $\sigma = sign((pk_u, v), sk_u)$

Validate $pk_u \notin M[u]$
Validate $verify((pk_u, v), \sigma, pk_u) = 1$
Validate $B[u] \geq v$
$B[u] = B[u] - v$
Add pk_u to M
Send $\sigma_i = sign((pk_u, v), s_i) = H(pk_u, v)^{s_i}$ to user

Upon Receiving $(T_1, T_2, (pk_a, v_a), (pk_u, v_u), \sigma_{u_1}, \sigma_{u_2})$

Validate $T_1, T_2 \notin N$
Validate $verify((pk_{u_1}, v_1), \sigma_1, PK) = 1$
Validate $verify((pk_{u_2}, v_2), \sigma_2, PK) = 1$ if $T_2 \neq \perp$
Validate $verify((T_1, T_2, (pk_a, v_a), (pk_u, v_u)), \sigma_{u_1}, pk_{u_1}) = 1$
Validate $verify((T_1, T_2, (pk_a, v_a), (pk_u, v_u)), \sigma_{u_2}, pk_{u_2}) = 1$ if $pk_{u_2} \neq \perp$
Check $v_1 + v_2 = v_a + v_u$ ($v_2, v_u = 0$ if $\perp$)
Send $\sigma_{i_a} = sign((pk_a, v_a), s_i) = H(pk_a, v_a)^{s_i}$ to user
Send $\sigma_{i_u} = sign((pk_u, v_u), s_i) = H(pk_u, v_u)^{s_i}$ if not $\perp$ to user
Add T_1, T_2 to N

Upon Receiving Non-Interactive $(T_1, T_2, (pk_a, v_a), (pk_u, v_u), \sigma_{u_1}, \sigma_{u_2})$

Validate $T_1, T_2 \notin N$
Validate $verify((pk_{u_1}, v_1), \sigma_1, PK) = 1$
Validate $verify((pk_{u_2}, v_2), \sigma_2, PK) = 1$ if $T_2 \neq \perp$
Validate $verify((T_1, T_2, (pk_a, v_a), (pk_u, v_u)), \sigma_{u_1}, pk_{u_1}) = 1$
Validate $verify((T_1, T_2, (pk_a, v_a), (pk_u, v_u)), \sigma_{u_2}, pk_{u_2}) = 1$ if $pk_{u_2} \neq \perp$
Check $v_1 + v_2 = v_a + v_u$ ($v_2, v_u = 0$ if $\perp$)
$B[a] = B[a] + v_a$ ▷ That's the only difference
Send $\sigma_{i_u} = sign((pk_u, v_u), s_i) = H(pk_u, v_u)^{s_i}$ if not $\perp$ to user
Add T_1, T_2 to N

1.2 Properties Proofs

1.2.1 Unforgeability Proof

- To forge a token, the adversary needs a valid threshold signature under the shared secret key $g(0)$
- Because g is a polynomial of degree f, any set of f or fewer points (shares) does not provide information about $g(0)$
- To compute it, the adversary must obtain $f + 1$ valid signature shares.
- By the protocol, each server issues a signature share only after executing one of the protocol operations, under the condition to
  - Correct signatures under the provided keys
  - Value consistency
  - Non-spent tokens
- In any case, omission-faulty servers may remain silence, but never issue incorrect or unauthorized shares
- Thus, the adversary cannot obtain $f + 1$ valid shares unless at least one of the protocol operations completes successfully
- Therefore, producing a valid token without server issuance is impossible □

1.2.2 Authorization Unforgeability

- A token (pk, v, σ) is spent if it is added to the nullifiers set N
- By the protocol, to be added to N, it should pass all the validation checks in the Pay operations
- Without producing a valid authorization signature under the public key embedded in that token, pk, the adversary will never pass the check in line 14 □

1.2.3 No Double Spend

- Assume that a valid token $T_1 = (pk, v, \sigma)$ is successfully spent in two different transactions
- By the protocol, for each successful transaction, at least $f + 1$ servers must:
  - Accept all the validation checks
  - Add T_1 to the nullifier set N
- Let A be the set of servers that accepted and signed the first transaction, B the second
- $|A| \geq f + 1$, $|B| \geq f + 1$
- $|A \cap B| = |A| + |B| - |A \cup B| \geq f + 1 + f + 1 - (2f + 1) = 1$
- This means that there is at least 1 server in both sets
- Because we deal with omission failures, we know this 1 server is not faulty (otherwise he would not send a message)

- By the protocol, the first time T_1 arrives the server, it signs it and adds it to N
- The second time T_1 arrives, it is already in N, hence, the transaction must be denied 11
- Contradiction to the assumption □

1.2.4 Liveness

- The network is synchronous with a known delay limit Δ
- In all operations, the user waits for responses from $f+1$ servers in order to complete the action
- The user sends his request to all $2f+1$ servers
- At most f server may be faulty and omit the request
- Therefore, at least $f+1$ servers will receive the request and send a response
- The user will reconstruct the final token and complete its operation successfully, in total of 2 rounds □

1.2.5 Conservation of Value

- $B[u]$ - un-mint balance for user u
- τ - set of all the unspent tokens
- For a token $T = (pk, v, \sigma)$, let $val(T) = v$
- Define:
$$\Psi = \sum_{u \in U} B[u] + \sum_{T \in \tau} val(T) \tag{1}$$
- Let's show that every operation preserves Ψ

Initial State

- No tokens
- Each user has an initial balance of 100
- $$\Psi_{init} = \sum_{u \in U} 100 \tag{2}$$

Mint

- Server decreases balance $B[u] = B[u] - v$
- Server issues a token $T = (pk, v, \sigma)$
- $$\Psi = 100 - v + \sum_{u' \in U, u' \neq u} 100 + val(T) = \Psi_{init} \tag{3}$$

Interactive Pay

- Before the operation we have T_1, T_2
- $$\Psi_{before} = v_1 + v_2 + \sum_{u \in U} B[u] + \sum_{T \in \tau, \neq T_1, T_2} val(T) \tag{4}$$
- Server issues two tokens $T_a = (pk_a, v_a, \sigma_a)$, $T_b = (pk_b, v_b, \sigma_b)$ s.t. $v_1 + v_2 = v_a + v_b$
- $$\Psi = v_a + v_b + \sum_{u \in U} B[u] + \sum_{T \in \tau, \neq T_a, T_b} val(T) = \Psi_{before} \tag{5}$$

Non-Interactive Pay

- Before the operation we have T_1, T_2
- $$\Psi_{before} = v_1 + v_2 + \sum_{u \in U} B[u] + \sum_{T \in \tau, \neq T_1, T_2} val(T) \tag{6}$$
- Server issues a token $T = (pk, v, \sigma)$ and increases $B[u'] = B[u'] + v'$ s.t. $v_1 + v_2 = v + v'$
- $$\Psi = v + v' + \sum_{u \in U, \neq u'} B[u] + \sum_{T \in \tau, \neq T} val(T) = \Psi_{before} \tag{7}$$

Conclusion Every operation preserves the total value □

2 Verifiable Secret Sharing

2.1 Protocol Pseudo-code

Algorithm 3 Share

Dealer

Input is s
Choose a random polynomial g of degree f s.t. $g(0) = s$: $g(x) = s + a_1x + a_2x^2 + ... + a_fx^f$
For each party i send $< E_i(g(i)) >$
Upon receiving EMPTY(i), broadcast $< E_i(g(i)) >$

Party

if Received $< E_i(g(i)) >$ from dealer within Δ time **then**
    Calculate $g(i)$
else
    broadcast EMPTY(i)
end if
Save all the EMPTY(j) messages in *empties*
Broadcast all the envelopes you have
Save all the $< E_j(g(j)) >$ you hear *envelopes*
if Exists j s.t. $j \in$ *empties* and $j \notin$ *envelopes* **then**
    Output $\perp$
end if

Algorithm 4 Recover

Party

If you outputted $\perp$ in *share* phase, output $\perp$
If you have $g(i)$, send it to everyone
Upon receiving $f+1$ shares, use Lagrange interpolation to reconstruct the shares to the final $g(0) = s$

2.2 Properties Proofs

2.2.1 Termination

1. **Termination of** *share*

    - **Round 1** Non-faulty parties invoke the protocol and wait one round for a share
    - **Round 2** Non-faulty parties who did not get a share broadcast EMPTY
    - **Round 3** Dealer either sends missing shares or stays silence
    - **Round 4** Non-faulty parties echos their envelopes. If the dealer sends to even one honest, all will get it. If it remains silence, all will output $\perp$
    - Total of 4 rounds and $O(n^2)$ messages □

2. **Termination of** *recover*

    - **Round 1** Non-faulty parties broadcast their share

- **After Round 1** Non-faulty parties interpolates $g(0)$
- Total of 1 round and $O(n^2)$ messages □

2.2.2 Hiding

- The adversary only sees f points from the f faulty servers
- It cannot see other points because of the envelopes
- The polynomial is of degree f, which means that the adversary needs $f + 1$ points to compute s
- That's is why it cannot distinguish between the real world (with a secret) or a random simulated world □

2.2.3 Validity

- Since the dealer is honest, after the *share* phase each party i has his share $g(i)$
- We have $2f + 1$ parties, at most f faulty
- We have at least $f + 1$ honest parties that will broadcast their share
- Using $f + 1$ points and Lagrange interpolation we can reconstruct the secret s □

2.2.4 Binding

- Let E be the extractor, given the views of all non faulty parties at the termination of *share*
- If E sees that the dealer did not respond to EMPTY complaints, it outputs $b = \perp$
- Due to the last echo round in the *share* phase, if one honest party sees an undressed EMPTY message, all parties see it, and output $\perp$
- If E sees that all the honest parties received their shares, it outputs $b = g(0)$
- Each of the honest parties will broadcast his share, and they will reach at least $f+1$ shares, due to $Validity$, and calculate $g(0)$ □

3 Distributed Key Generation and Agreement

3.1 Protocol Pseudo-code

Algorithm 5 DKG

Party i

1: Randomly choose s_i
2: Run a reliable broadcast on the public key share h^{s_i}
3: After all RBs terminate, hold a set B of all the parties you successfully received their broadcast
4: In parallel, run VSS where you are the leader and s_i is the input, and participate in others' VSSs
5: After all VSSs terminate, hold a set V of all the parties that finished VSS successfully (did not output $\perp$)
6: $H = B \cap V$ - the honest parties from both protocols
7: The secret share is

$$S_i = \sum_{j \in H} g_j(i) \tag{8}$$

8: The public key is

$$PK = \prod_{j \in H} h^{s_j} = h^{\sum_{j \in H} s_j} \tag{9}$$

3.2 Properties Proofs

3.2.1 Termination

- All the Reliable Broadcast protocols run in parallel, each of them terminates after 4 rounds
- All the VSS protocols run in parallel, each of them terminates after 4 rounds
- Other steps are local and mathematical
- Because the network is synchronized, all the non-faulty parties terminated together
- The total round complexity is 4 and the message complexity is n parties times $O(n^2)$ so $O(n^3)$ □

3.2.2 Hiding

- Due to discrete logarithm problem, h^{s_i} does not reveal s_i
- VSS satisfies hiding, so for each VSS, the adversary can learn up to f points
- It is not enough to reconstruct the secret, s_i is uniformly random to the adversary
- $SK = \Sigma s_i$, where at least $f + 1$ elements are uniformly random to the adversary
- So also SK is uniformly randomly distributed, and the advesary learns nothing □

3.2.3 Validity and Agreement

- Validity and agreement of Reliable Broadcast ensures that if one honest party has $i \in B$, then all honests have
- Validity of VSS ensures that if one honest party has $i \in V$, then all honests have
- Therefore, every honest party computes the exact same intersection set H
- Since everyone has the same H, their PK is the same
- From the linearity of the exponent $h^{SK} = PK$ □

3.2.4 Validity and Liveness

- If party i is honest, it will successfully finish RB and VSS, therefore $i \in H$ for every party
- Since there are at least $f + 1$ honest, we can assure that $|H| \geq f + 1$
- For every party in $j \in H$, a honest party i is guaranteed to have $g_j(i)$, so he can successfully compute his share S_i
- We will have at least $f + 1$ to restore the secret □

4 Bonus

4.1 Payment Scheme

- In synchronous mode with Byzantine adversary and PKI, we can reach agreement when $n = 2f + 1$ so it does not need to be changed!
- Using the PKI, each entity signs its entire payloads, so messages cannot be forged
- Also, each entity verifies the payload with the signature
- Servers add another round, after each time they receive a message from the user, of equivocation detection - broadcast the signed payload and make sure the user did not send different message to different servers

□

4.2 Verifiable Secret Sharing

- In the byzantine mode, the adversary can send points that do not lie on the same polynomial
- I saw online that there is Feldman's method:
  - The dealer also publicly signs and sends a commitment, that is some group member h to the power of the polynomial coefficients:
  - $[C_0 = h^s, C_1 = h^{a_1}, \ldots, C_f = h^{a_f}]$
  - When a party gets his share $g(i)$, it calculates $h^{g(i)}$ and then validates:

$$C_0 \cdot C_1^i \cdot C_2^{i^2} \cdot \ldots \cdot C_f^{i^f} = h^s \cdot h^{a_1 i} \cdot h^{a_2 i^2} \cdot \ldots \cdot h^{a_f i^f} = h^{s + a_1 i + a_2 i^2 + \ldots + a_f i^f} = h^{g(i)} \quad (10)$$

- Of course all messages are signed, and all messages you receive you verify
- Thanks to the commitment, during the *recover* phase, honest parties can discard shares that do not pass the commitment check

□

4.3 Distributed Key Generation and Agreement

- Of course all messages are signed, and all messages you receive you verify
- We will use the upgraded VSS with the commitment
- The public key broadcasting will remain the same thanks to the PKI

□